C.2.1 Storing key-value pairs

We assume keys and values can be picked from different domains. Then, we need to decide if duplicate keys are allowed. For sake of simplicity, we will consider only sets of unique keys. It is always possible to make a static group of keys unique.

The easiest case is when keys are integers: in theory you can use an array to store the value associated with key `k` in the `k`-th element of the array. This, however, is only possible when the possible keys are limited in range: if, for example, any 32-bit positive integer was allowed, we would need an array with more than 3 billion elements - possibly larger than even the most modern machine's RAM. Even worse if we considered "longs", 8-byte integers: we'd be talking about 18 billion of billions of elements. No, it's not a typo: we are talking about billions of billions.

The worst part is that, by using arrays, even if we know that we'll store only a few thousands integer keys, we'll need an array with 2^{32} elements if they can be any integer value.

If we do have to store that many elements, there is little we can do to improve. Sometimes, however, we know that we might store only a handful of elements (say, a few hundreds or thousands). Even if we can choose these elements from a large set (for example all the integers that can be represented with 32 bits, for a total of around 4 billion elements). If that's the case, then we can do better. That's precisely where hashing comes to the rescue.

C.2.2 Hashing

Hashing provides a compromise between key-indexed arrays and unsorted arrays in conjunction with sequential search. The former solution offers constant time search but needs space proportional to the set of possible keys. The latter requires linear time for search but space proportional to the number of actual keys.

Using hash tables, we fix the size of the array to, say, `M` elements. As we'll see, we might store more than `M` elements though, depending on how we